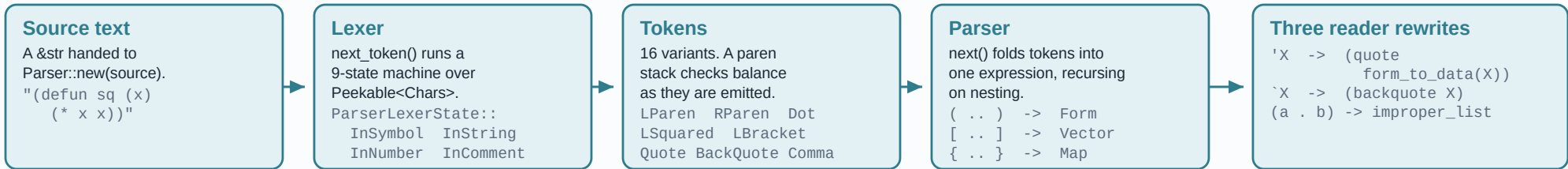


How an expression becomes a value

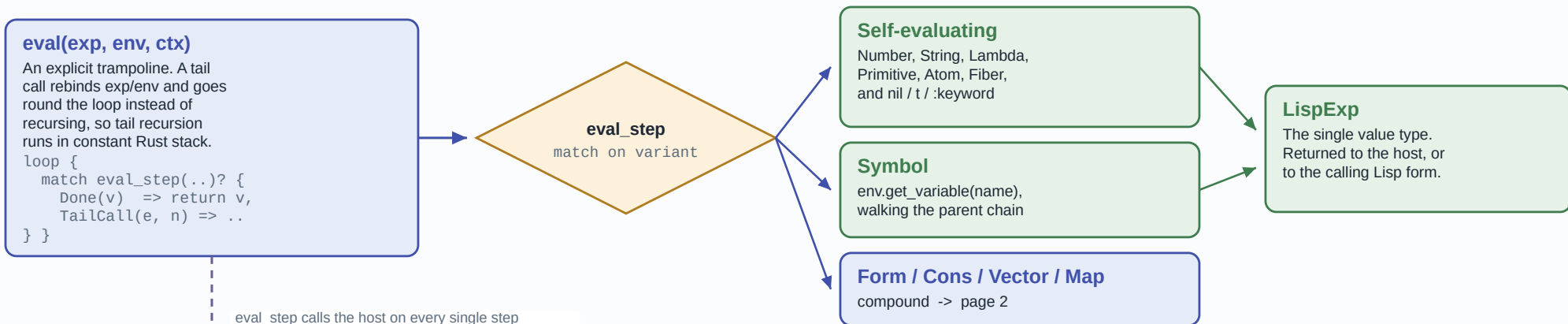
The whole path, once: source text to a token stream to a syntax tree, then one trampoline that reduces that tree to a LispExp.

READ core/src/lisp/lisp.rs :: Parser

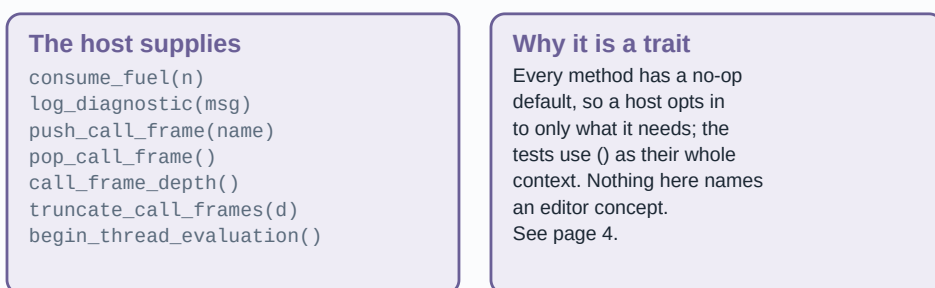


one LispExp per top-level form

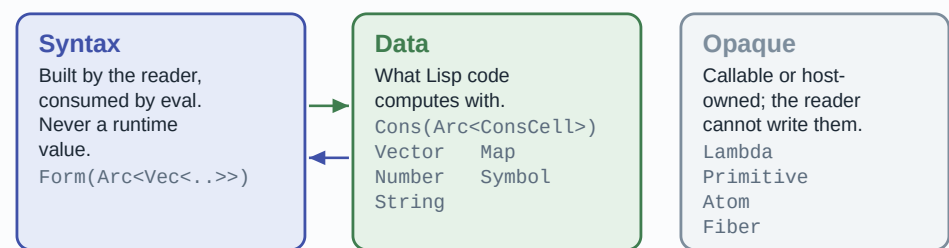
EVALUATE eval() -> eval_step()



HOST BOUNDARY trait LispContext



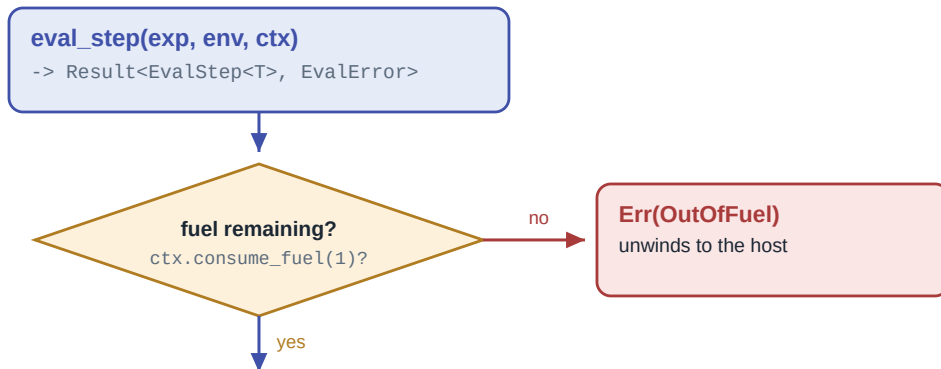
THE VALUE TYPE enum LispExp<T: LispContext>



form_to_data and data_to_form are the only bridges between the two -
a Form in a value position, or a Cons in eval position, is a bug. (Form was named List before the rename.)

eval_step - one turn of the trampoline

Every reduction passes through this match exactly once. Compound expressions hand back a TailCall; everything else is Done.



match exp {	=> EvalStep
Number String Lambda Primitive Atom Fiber	Done(exp.clone()) self-evaluating
Symbol("nil" "t") Symbol(":keyword")	Done(exp.clone()) self-evaluating too
Symbol(name)	env.get_variable(name) Some(v) => Done(v) None => Err(UnboundVariable)
Cons(..)	TailCall(data_to_form(exp)?, env) data reaching eval position: (eval '(+ 1 2)), macro output
Vector(v)	eval every element Done(Vector)
Map(m)	eval every value Done(Map)
Form(items)	the only variant that can call something -> right

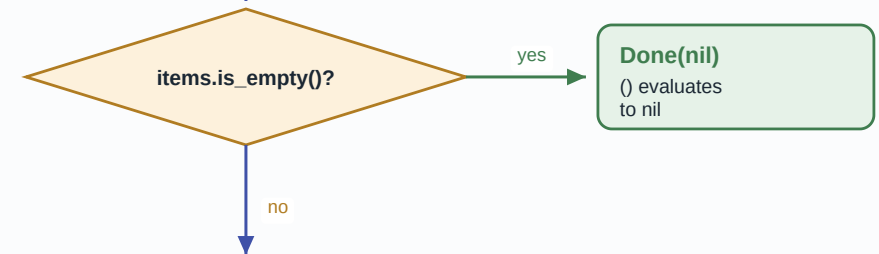
Two things worth noticing

The fuel check is the first statement, before the match, so every reduction step is charged once whichever branch runs - loops, calls and argument evaluation are all metered by construction. And Cons reaching here is not an error: it is how data becomes code, and the only place data_to_form runs.

Form(items) - the call path

Form(items)

A vector-backed syntax node:
items[0] is the head, the rest are arguments.



items[0]	action
Symbol(s)	eval_special_form_or_call_step(s, &items[1..], env, ctx) 22 special forms, then macro, then function -> page 3
Form(..)	evaluate the head first, splice the result back in front of the arguments, then TailCall the rebuilt form - ((f) 1 2)
Lambda(l)	a literal lambda in head position: eval the args, then call it directly. Same body protocol as page 3.
-	Err(InvalidFunctionCall)

Why every branch returns EvalStep, never a value

A branch that has finished returns Done. A branch whose result is whatever some other expression evaluates to returns TailCall(next_exp, next_env) instead of calling eval recursively - eval() then rebinds its two locals and goes round the loop again.

So a tail-recursive Lisp function runs in one Rust stack frame however deep it goes, and the depth limit is the fuel budget rather than the native stack.

Calling something

A head symbol is tried as a special form, then as a macro, then as a function. Each layer has a different rule about when arguments get evaluated.

eval_special_form_or_call_step(symbol, args, env, ctx)

25 special forms, matched by name

Their arguments are NOT evaluated first - each arm decides what to evaluate, and when. That is exactly what makes them special forms.

quote	backquote	if	cond	when
unless	and	or	progn	while
dolist	dotimes	let	let*	setq
defvar	defconst	defun	lambda	defmacro
unwind-protect	catch	condition-case	spawn	fiber

no special form matched

eval_macro_or_function_call_step(symbol, args, env, ctx)

env.get_macro(symbol)
a macro with this name?

yes

no

eval_function_call_step(symbol, args, env, ctx)

The arguments are evaluated first, strictly, left to right - then:

env.get_function(symbol) =>

Some(Lambda l)	bind and run the body -> right
Some(Primitive)	call the fn pointer -> right
Some(other)	Err(IncorrectFunctionDefinition)
None	Err(UndefinedFunction(symbol))

Macro: expand now, evaluate the expansion after

```
let frame = Env::new_child(&m.env);  
bind_lambda_args(&m, args, &frame?); // args RAW, unevaluated
```

```
let mut expansion = nil;  
for form in &m.body {  
    expansion = eval(form, frame.clone(), ctx?);  
}  
TailCall(expansion, env) // the CALLER's env
```

No call frame is pushed, so macros never appear in a backtrace, and the body is re-expanded on every call - there is no expansion cache.

Function: evaluate the arguments, then enter the body

```
let args = args.map(|a| eval(a, env.clone(), ctx?)); // strict  
ctx.push_call_frame(symbol);
```

```
Lambda let frame = Env::new_child(&l.env); // lexical: l.env  
bind_lambda_args(&l, &args, &frame?);  
for f in &l.body[..n-1] { eval(f, frame.clone(), ctx?); }  
ctx.pop_call_frame(); // BEFORE the tail call  
TailCall(l.body.last(), frame)
```

```
Primitive let r = pointer(&args, env.clone(), ctx?);  
ctx.pop_call_frame(); Done(r)
```

Three consequences of that ordering

1. The frame is popped before the tail call, so a function in tail position never shows up in a backtrace. This is why the backtrace docstring warns that a failure in a tail call reports an apparently shallow stack.
2. The frame is pushed after the arguments are evaluated, so an error raised while computing an argument is attributed to the caller, not the callee.
3. On the error path the ? short-circuits and the frame is deliberately left on the stack - that is what the backtrace reads. `truncate_call_frames()` is how a recovering caller such as `eval-string-safe` puts the stack back.

Failure and non-local exit

Two ways out of a computation that is not returning normally. They are kept apart on purpose: one is a control transfer, the other is a failure.

THE ERROR TYPE `enum EvalError<T: LispContext>`

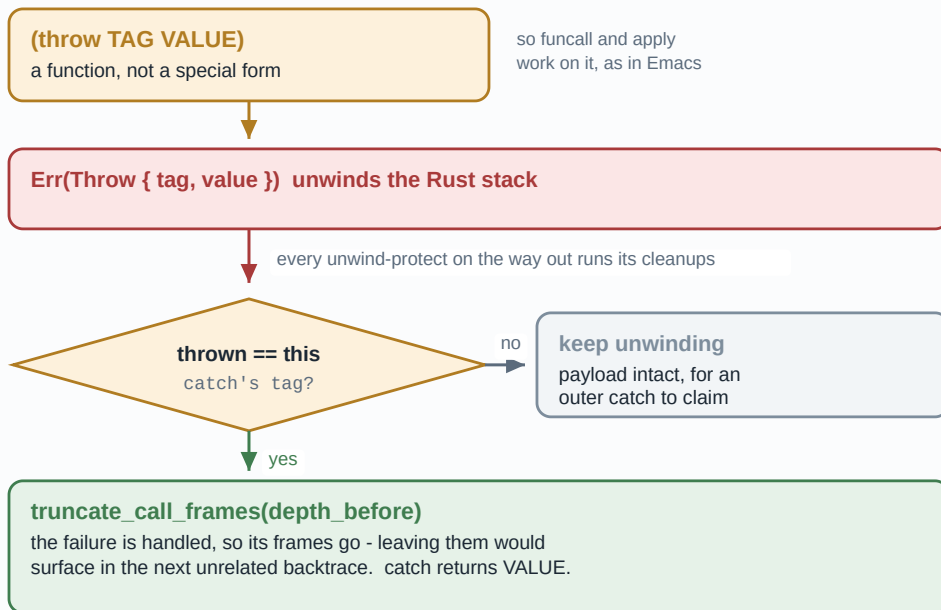
group	variants (31 in total)
Runtime failures	UnboundVariable UndefinedFunction UnvalidFunctionCall WrongNumberOfArguments WrongArgumentType
Malformed forms	QuoteNotOneArgument LetUnvalidBindingAt CondInvalidClause DefunMisplacedParamMarker ... a dozen more
Non-local exit	Throw { tag, value } Signal { symbol, data }
Host	OutOfFuel RuntimeMessage(String)

Why it is generic over T

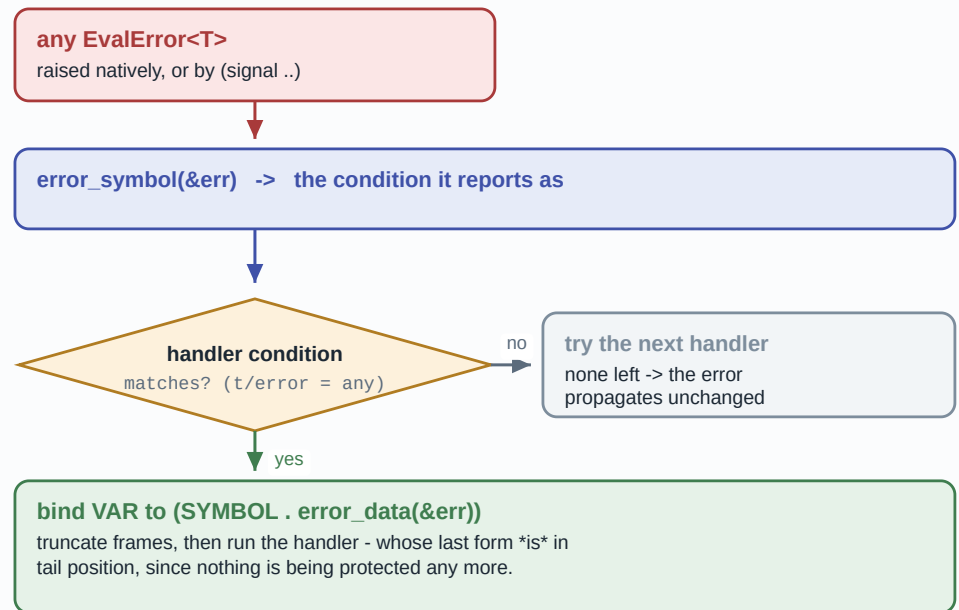
So the error can carry Lisp *values*. A thrown payload, and the offending value in a type error, are both `LispExp<T>` - only expressible if the error knows what T is.

Around forty sites used to render that value with `format!("{:?}", v)` and throw it away. Fine while errors only reached Rust; useless once a handler in Lisp receives them.

CONTROL TRANSFER `(throw TAG VALUE) -> (catch TAG ..)`



FAILURE `any EvalError -> (condition-case VAR ..)`



The rule that makes both usable:

a condition-case never intercepts a throw aimed past it, and a catch never intercepts an error that belongs to a handler further out.

```
(catch 'tag (condition-case nil (throw 'tag 11) (error 'wrong))) => 11
(catch 'tag (undefined-fn)) => propagates UndefinedFunction
```

The machinery around the loop

Four concerns that cut across every evaluation: where names resolve, what stops a runaway program, what a backtrace is made of, and what happens off the main thread.

1 SCOPE Env - three namespaces, one parent chain

struct Env<T>

```
variables: RwLock<HashMap<.,>>
functions: RwLock<HashMap<.,>>
macros:   RwLock<HashMap<.,>>
parent:   Option<Arc<Env<T>>>
```

Three separate maps make this a Lisp-2:
(f x) and x can name different things.

get_variable / get_function / get_macro
each walk the parents up to the root.

call frame

```
Env::new_child(&l.env)
```

parent

closure env

```
captured in Lambda.env
```

parent

root env

```
setup_base_env()
```

A Lambda captures its Arc<Env> when defined:
calls are lexically scoped, never dynamically.

The one cycle in the design

A defun stores the Lambda in the same Env the Lambda holds an Arc to, so every top-level function closes a reference cycle Arc cannot collect. Harmless for the global env, which outlives the process; a real leak only for a closure kept in a let frame, or an Atom.

2 METERING FuelMeter - shared budget, per-thread counter

FuelMeter (shared)

Lives in the host and is cloned with it into any thread it spawns.

```
budget: AtomicU32
```

thread_local! (per thread)

const-initialised, so a read is one %fs-relative load.
Armed by arm_thread().
FUEL / DEPTH: Cell<u32>

call	effect
fuel.begin()	if DEPTH == 0 { FUEL = budget } DEPTH += 1 -> FuelScope guard
consume(n)	FUEL.checked_sub(n) None => Err(Exhausted)
drop(scope)	DEPTH -= 1

Refilling only at depth 0 is the point: a nested evaluation cannot launder itself a fresh budget.

3 DIAGNOSTICS the call-frame stack behind (backtrace)

push_call_frame(s)	on entry, after the args
pop_call_frame()	on success, before a tail call
(error path)	deliberately does NOT pop
call_frame_depth()	-> usize, saved beforehand
truncate_call_frames	(d) restore after recovery

after an error

```
minibuffer-confirm
funcall
eval-expression-
confirm
eval-string
```

innermost first;
tail calls absent;
(backtrace) drops [0]

Why frames survive the error

The ? that propagates an EvalError skips every pop on the way out, so the stack is left standing exactly as it was at the point of failure - that is what report-error reads. A caller that swallows an error instead of propagating it must therefore put the stack back itself, which is what eval-string-safe does.

4 CONCURRENCY (spawn) and (fiber)

(spawn LAMBDA) - a real OS thread

```
let mut thread_ctx = ctx.clone(); // every field is an Arc
std::thread::spawn(move || {
    thread_ctx.begin_thread_evaluation(); // arm this thread
    for e in &l.body { eval(e, frame.clone(), &thread_ctx)?; }
});
Done(nil) // does not join
```

Values cross the boundary because every LispExp payload is an Arc, and Env guards its maps with RwLock.

(fiber ..) / (resume f) / (reset f) - cooperative, same thread

SharedFiber(Arc<RwLock<FiberState>>) holds a body, its env and an is_done flag.
resume runs the next body form and hands back its value; reset rewinds it. No thread, no preemption - the interpreter simply stops between forms.